

คู่มือโปรแกรมเมอร์ Embedded Linux System
Programmer Manual for Embedded Linux System

โดย

นายเกษร มฤกุล
นายสถาพร สงวนวงษ์

อาจารย์ที่ปรึกษา

รศ. ดร. บุญธีร์ เครือตราชู
ผศ. กฤตวัน ศิริบูรณ์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
ภาควิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2547

สารบัญ

	หน้า
1. การติดตั้ง Debian Linux	3
2. การปรับแต่งการใช้งาน	3
2.1 การ Compile kernel	3
2.2 การปรับแต่ง Service ต่างๆ	3
3. อธิบายการทำงานของโปรแกรม	8
3.1 โครงสร้างของโปรแกรม	8
3.2 ส่วน Header	10
3.3 ส่วน Procedure	13
4. การ Compile Code	32
5. การ run / stop program	32
6. การทำ Startup script	32

1. การติดตั้ง Debian Linux

การติดตั้ง Debian Linux ด้วย kernel 2.4 สามารถทำได้โดยพิมพ์คำสั่ง bf24 หลังจาก boot เครื่องคอมพิวเตอร์ด้วยแผ่น cdrom และในส่วนรายละเอียดของขั้นตอนการติดตั้งสามารถ download ได้ที่

<http://www.debian.org/releases/stable/i386/install.en.pdf>

2. การปรับแต่งการใช้งาน

2.1 การ Compile kernel

การ compiled kernel เป็นส่วนการปรับแต่งเพิ่มเติมอาจต้องทำเพื่อให้รองรับการทำงาน mainboard รุ่นอื่นๆ แต่จากการทดสอบของผู้จัดทำพบว่าการนำ Debian Linux kernel 2.4 ไปใช้งานกับ mainboard ตระกูล x86 ไม่จำเป็นต้อง compiled kernel ใหม่ โดยสามารถใช้งานได้เป็นปกติ หากผู้ใช้ต้องการ compiled kernel ใหม่สามารถดูรายละเอียดได้จาก www.kernel.org

2.2 การปรับแต่ง Service ต่างๆ

2.2.1 ตรวจสอบ version ของ linux kernel และ linux include file

เนื่องจากการพัฒนาโปรแกรมบน linux ในระหว่าง run โปรแกรมระบบปฏิบัติการจะตรวจสอบ version ของโปรแกรมที่กำลังทำงานด้วยว่าเป็น version เดียวกับ kernel หรือไม่ ดังนั้นการตรวจสอบ version ของ kernel และ include file จึงเป็นสิ่งจำเป็นสามารถตรวจสอบได้โดย

- ตรวจสอบ version ของ linux kernel โดยพิมพ์คำสั่ง

```
uname -a
```

```
Linux project 2.4.18 #2 SMP Thu Sep 30 07:15:47 ICT 2004 i586 unknown
```

- ตรวจสอบ version ของ include file

```
cat /usr/include/linux/version.h
```

```
#define UTS_RELEASE "2.4.18"
```

```
#define LINUX_VERSION_CODE 132114
```

```
#define KERNEL_VERSION(a,b,c) (((a) << 16) + ((b) << 8) + (c))
```

ในส่วนที่ขีดเส้นใต้จะต้องตรงกันทั้ง 2 จุด ถ้าไม่ตรงให้ทำการแก้ไขค่าใน file `/usr/include/linux/version.h` ให้ตรงกับ version ของ kernel ที่ใช้อยู่

2.2.2 ตรวจสอบการทำงานของ network card

ในเบื้องต้นจะต้องตรวจสอบว่า network card ทำงานอยู่ และ กำหนดหมายเลข IP อย่างถูกต้องหรือไม่ ซึ่งหมายเลข IP นี้ผู้ใช้งานสามารถเปลี่ยนแปลงและกำหนดใหม่ได้แต่ควรสอดคล้องกับ client ที่ใช้ในการพัฒนาโปรแกรม ตามตัวอย่างได้มีการกำหนดหมายเลข IP ไว้ที่ 192.168.0.50 คำสั่งที่ใช้ตรวจสอบคือ `ifconfig` หน้าจอจะแสดงรายชื่อของ network card ที่กำลังทำงานอยู่ทั้งหมด

```
eth0  Link encap:Ethernet HWaddr 00:50:04:BA:99:34
      inet addr:192.168.0.50 Bcast:192.168.0.255 Mask:255.255.255.0
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:961 errors:0 dropped:0 overruns:0 frame:0
      TX packets:619 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:100
      RX bytes:59476 (58.0 KiB) TX bytes:58146 (56.7 KiB)
      Interrupt:11 Base address:0xe000

lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      UP LOOPBACK RUNNING MTU:16436 Metric:1
      RX packets:83 errors:0 dropped:0 overruns:0 frame:0
      TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:5815 (5.6 KiB) TX bytes:5815 (5.6 KiB)
```

ผู้ใช้ควรให้ความสนใจเฉพาะ network card `eth0` เพราะ `lo` เป็น local interface ของระบบ ถ้าปรากฏข้อความดังตัวอย่างด้านบนแสดงว่า network card ถูกติดตั้งและทำงานอย่างสมบูรณ์แล้ว ถ้าไม่เป็นไปตามที่กล่าวมาให้ทำการตรวจสอบค่าภายใน file `/etc/network/interfaces` ภายใน file ควรประกอบด้วย

```
# /etc/network/interfaces -- configuration file for ifup(8), ifdown(8)

# The loopback interface
auto lo eth0
iface lo inet loopback
iface eth0 inet static
    address 192.168.0.50
    netmask 255.255.255.0
```

ส่วนที่ขีดเส้นใต้เป็นหมายเลข IP ของ server เมื่อแก้ไขเรียบร้อยแล้วให้ทำการ save และ พิมพ์คำสั่ง `/etc/init.d/network restart` เพื่อให้ระบบ network ทำงานใหม่อีกครั้ง

2.2.3 ตรวจสอบการทำงานของ telnet และ ftp

ftp และ telnet เป็นสิ่งจำเป็นที่ใช้ในระหว่างพัฒนาระบบ เพราะการทำงานส่วนใหญ่จะกระทำที่เครื่องอื่นที่ไม่ใช่ server ดังนั้นจะต้องมีการ upload และทดสอบโปรแกรมที่ส่งไปตลอดเวลา

การตรวจสอบว่า ftp service ทำงานอยู่หรือไม่สามารถทำได้โดยพิมพ์คำสั่ง `ftp localhost` จะต้องปรากฏข้อความด้านล่างถ้า ftp กำลังทำงานอยู่

```
Connected to project.
220 project FTP server (Version 6.4/OpenBSD/Linux-ftpd-0.17) ready.
Name (localhost:root):
```

จะต้องปรากฏ login prompt ดังตัวอย่างด้านบน และเช่นเดียวกับ ftp ในการตรวจสอบ telnet service สามารถทำได้เช่นเดียวกันโดยพิมพ์คำสั่ง `telnet localhost` จะต้องปรากฏข้อความดังนี้

```
Trying 127.0.0.1...
Connected to project.
Escape character is '^]'.
Debian GNU/Linux 3.0 project
project login:
```

ข้อความด้านบนจะถูกแสดงในกรณีที่ telnet และ ftp กำลังทำงานอยู่ ถ้าไม่เป็นไปตามตัวอย่างให้ทำการตรวจสอบ file /etc/inetd.conf ว่าทำการเปิดให้บริการ ftp และ telnet หรือยัง โดยภายใน inetd.conf ควรจะเป็นดังรูป

```
# /etc/inetd.conf: see inetd(8) for further informations.
```

```
#:STANDARD: These are standard services.
```

```
ftp      stream tcp      nowait root           /usr/sbin/tcpd
          /usr/sbin/in.ftpd

telnet   stream tcp      nowait telnetd.telnetd /usr/sbin/tcpd
          /usr/sbin/in.telnetd
```

สังเกตว่าหน้าบรรทัดที่มีข้อความ ftp และ telnet จะต้องไม่มีเครื่องหมาย “#” อยู่ด้านหน้า ถ้ามีให้ทำการลบเครื่องหมาย “#” ออกจากนั้นทำการ save และ restart ระบบใหม่ และทำการตรวจสอบดังที่กล่าวมาแล้วอีกครั้งถ้ายังไม่สามารถใช้งานได้ให้ตรวจสอบว่า ftp และ telnet ได้ถูกติดตั้งหรือยังโดยการตรวจสอบ package ที่ติดตั้งไว้ดังนี้

```
dpkg -l |grep ftp
```

```
ii ftp      0.17-9      The FTP client.
ii ftpd     0.17-13     FTP server
ii lftp     2.4.9-1     Sophisticated command-line FTP/HTTP client p
```

```
dpkg -l |grep telnet
```

```
ii telnet   0.17-18     The telnet client.
ii telnetd  0.17-18     The telnet server.
```

ถ้าไม่เป็นไปตามตัวอย่างด้านบนที่กล่าวมาให้ทำการติดตั้ง package ใหม่โดยถ้า server สามารถเชื่อมต่อ internet ได้ให้ทำการพิมพ์คำสั่ง apt-get install ftp เพื่อติดตั้งการให้บริการ ftp และ apt-get install telnet เพื่อติดตั้งการให้บริการ telnet ระบบจะทำการ download และติดตั้ง package โดยอัตโนมัติ

2.2.4 ตรวจสอบการทำงานของ web service

การทำงานของ web server หรือ apache สามารถตรวจสอบการทำงานได้โดยใช้คำสั่ง `ps ax|grep apache` จะปรากฏข้อความดังนี้

198 ?	S	0:00	/usr/sbin/apache
312 ?	S	0:00	[apache]
313 ?	S	0:00	[apache]
314 ?	S	0:00	[apache]
315 ?	S	0:00	[apache]
316 ?	S	0:00	[apache]
927 pts/0	S	0:00	grep apache

ถ้าไม่มีข้อความที่คล้ายคลึงกับตัวอย่างด้านบนให้ทำการตรวจสอบว่า apache ถูกติดตั้งแล้วหรือยัง โดยใช้คำสั่ง `dpkg -l |grep apache` จะปรากฏข้อความดังนี้

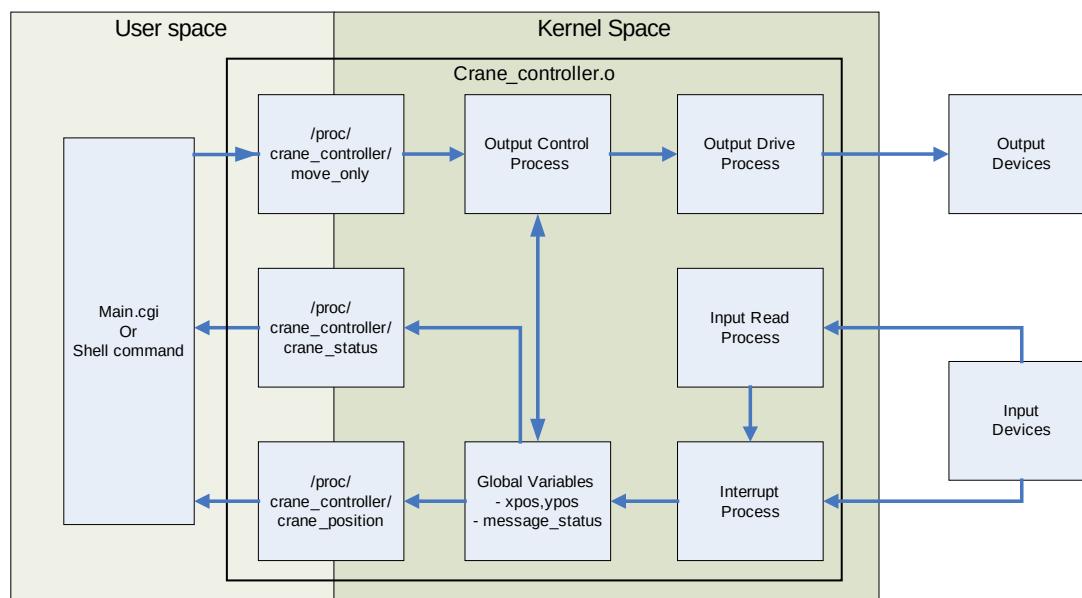
ii	apache	1.3.26-0woody3	Versatile, high-performance HTTP server
ii	apache-common	1.3.26-0woody3	Support files for all Apache web servers

ถ้าไม่ปรากฏข้อความดังตัวอย่าง ให้ทำการติดตั้ง apache โดยใช้คำสั่ง `apt-get install apache` หรือติดตั้งจากแผ่น CDROM Debian linux แผ่นที่ 1 ด้วยคำสั่ง `dpkg -i` ตามด้วยชื่อ file apache

3. อธิบายการทำงานของโปรแกรม

3.1 โครงสร้างของโปรแกรม

เนื่องจากโปรแกรมมีลักษณะเป็น module จึงมีโครงสร้างที่คาบเกี่ยวระหว่าง user space และ kernel space ของระบบปฏิบัติการ โดยส่วนที่อยู่ใน user space นั้นผู้ใช้จะสามารถมองเห็นได้โดยตรงและถูกใช้เป็นเส้นทางส่งและรับข้อมูลระหว่างผู้ใช้กับ module ที่กำลังทำงานอยู่ โครงสร้างหลักๆ ของโปรแกรมสามารถแสดงได้ดังรูป



รูปแสดงลักษณะโครงสร้างของ *Crane Controller device driver*

User interface (Main.cgi or shell command)

ทำหน้าที่ติดต่อกับผู้ใช้งาน เป็นส่วนรับคำสั่งจาก user แล้วส่งข้อมูล ไปยัง process file ในทางกลับกันยังทำหน้าที่รับข้อมูลจาก process file มาแสดงยัง user อีกทางหนึ่งด้วยโดยช่องทางการใช้งานระบบสามารถทำได้ 2 ทางคือ Linux shell command และ หน้า webpage main.cgi

/proc/crane_controller/move_only

เป็น process file ที่ถูกสร้างขึ้นขณะที่ module ถูก load เข้าสู่ kernel ทำหน้าที่เป็นส่วนรับค่าตำแหน่งของ crane จาก ผู้ใช้และส่งค่าตำแหน่งต่อไปยังส่วนควบคุมการหมุนของ stepping motor ภายในของ process ต่อไป

/proc/crane_controller/crane_position

เป็น process file ที่ถูกสร้างขึ้นขณะที่ module ถูก load เข้าสู่ kernel ทำหน้าที่นำค่าตัวแปร Global ตำแหน่งของ crane ไปแสดงยัง user space

/proc/crane_controller/crane_status

เป็น process file ที่ถูกสร้างขึ้นขณะโปรแกรมทำงาน ทำหน้าที่นำค่าตัวแปร Global ของสถานะ crane ไปแสดงยัง user space

output control process

เป็น process ที่อยู่ภายใน module ทำหน้าที่รับคำสั่งให้ stepping motor หมุนนำ crane ไปยังตำแหน่งที่ ผู้ใช้ต้องการ โดย process นี้จะเป็นตัวควบคุมทิศทางการหมุนตลอดจนความเร็วของ stepping motor ทั้งหมด โดยจะทำการตรวจสอบค่าตำแหน่งของ crane จากตัวแปร Global ตลอดเวลาเพื่อที่จะทราบว่า crane ไปถึงตำแหน่งที่ต้องการแล้วหรือยัง

output drive process

เป็นส่วนที่ทำหน้าที่จัดตำแหน่งข้อมูลที่จะส่งให้กับ stepping motor โดยจะใช้หลักการของการ masking bit ข้อมูลเพื่อกำหนดให้ค่าส่งไปยัง stepping motor ได้ถูกต้อง และหมุนได้อย่างถูกต้องโดยจะมีการอ่านค่าการหมุนจาก step table ในส่วนของตัวแปร Global

input read process

เป็นส่วนที่ทำหน้าที่รับสัญญาณ digital จากอุปกรณ์ภายนอกอย่างต่อเนื่อง process นี้จะต้องทำงานสอดคล้องกับ interrupt process เพราะเมื่อมีการเปลี่ยนแปลงของสัญญาณ input สัญญาณ interrupt จะต้องเปลี่ยนแปลงด้วย

interrupt process

เป็นส่วนที่ทำหน้าที่ทำงานตามสัญญาณ interrupt ที่เกิดขึ้นจากอุปกรณ์ภายนอกโดยสัญญาณ interrupt จะเกิดขึ้นพร้อมๆกับการเปลี่ยนแปลงของข้อมูล input ดังนั้นเมื่อเกิด interrupt ขึ้น จะต้องมีการอ่านค่าจาก input read process เพื่อที่จะทราบว่า เป็นสัญญาณ interrupt ของข้อมูลตัวใด

3.2 ส่วน Header

ส่วน include normal Header file ทั้งหมด

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/sched.h>
#include <asm/uaccess.h>
#include <linux/ioport.h>
#include <linux/interrupt.h>
#include <asm/io.h>
```

ส่วนกำหนดค่าคงที่

```
#define MODULE_VERSION      "1.0"
#define MODULE_NAME         "crane_controller"

/* All SPP PORT definitions */
#define SPPDATAPORT          0x378
#define SPPSTATUSPORT        (SPPDATAPORT + 1)
#define SPPCONTROLPORT       (SPPDATAPORT + 2)
#define INTERRUPT            7

/* SPP control port bit definitions */
#define OUTPUTENABLE          0x02
#define OUTPUTLATCH           0x04
#define INPUTENABLE           0x08
#define SPPPORTREAD           0x20
#define SPPINTERRUPTENABLE    0x10
#define MAXSTEP               3      /*max index of step table */
```

```

/* all delay time use in jiffies style */
#define MAXMOTORDELAY          1 /*used for delay motor driving */
#define MAXSENSORDELAY        250 /*used for delay x,y sensor input not occur within */
#define MINSENSORDELAY        200 /*used for delay x,y sensor input not occur within */
#define MAXINTERRUPTDELAY      1
#define MAXTIMEOUTDELAY        1500

/* SPP input bit definitions */
#define BOTTOMSW                0x01 /* bottom limit switch */
#define TOPSW                  0x02 /* top limit switch */
#define LEFTSW                 0x04 /* left limit switch */
#define RIGHTSW                0x08 /* right limit switch */
#define XPOSSENSOR             0x10 /* crane x position */
#define YPOSSENSOR             0x20 /* crane y position */

/* SPP output bit definitions */
#define XMOTORMASK              0x0F
#define YMOTORMASK              0xF0

```

ส่วนกำหนด Module description

```

MODULE_LICENSE("GPL");  MODULE_AUTHOR("Sthaporn
Sahounwong");  MODULE_DESCRIPTION("Crane controller process module");

```

ส่วนกำหนดโครงสร้างของ file structure ในรูปของ pointer

```

static struct proc_dir_entry *main_dir, /* main process directory */
    *move_only_file, /* move motor only process */
    *get_position_file; /* read crane position process */
    *get_status_file; /* read all crane status process */

```

ส่วนกำหนดตัวแปรที่ใช้ในโปรแกรมทั้งหมด โดยจะใช้งานแบบ Global variable

```

/*****
all status messages
*****/

/* crane status */
char *init_string = "Initializing crane please wait\n";
char *standby_string = "Crane standby for your request\n";
char *moving_string = "Crane moving to specified position\n";
char *demo_string = "Program running in demo mode\n";
char *timeout_error_string = "Crane not reach to target position in time\n";
char *left_limit_string = "Left limit action\n";
char *right_limit_string = "Right limit action\n";
char *top_limit_string = "Top limit action\n";
char *bottom_limit_string = "Bottom limit action\n";

/* user warning message */
char *user_position_error = "Please enter X,Y position between 0,0 and 5,5 coordinate\n";
char *user_keyin_error = "Please enter number in format X,Y between 0,0 and 5,5 coordinate\n";

/*****
/* define global variables here */
*****/

int xyposchange=1;    /*used for protect x,y position if motor not moving */
int timeout = 0;      /*used for limit motor action */
long xmotorstarttime = 0;    /* used for check x sensor interval time */
long ymotorstarttime = 0;
int xflag = 0;        /* start x sensor flag */
int yflag = 0;
int xflag2 = 0;
int yflag2 = 0;
int xpos=0;           /*crane x position counter */
int ypos=0;           /*crane y position counter */

```

```

char xdirect,ydirect;    /* global variable for motor direction */
char progmode = 'a';    /* a = actual , d = demo */
unsigned short motorout ;           /*variable for keep stepping motor data out */

unsigned char ystepable[MAXSTEP+1] = {0x03,0x06,0x0c,0x09};
unsigned char xstepable[MAXSTEP+1] = {0x90,0xc0,0x60,0x30};

int xindex=0;    /*stepper motor move index */
int yindex=0;

```

ส่วนกำหนด Procedure header ทั้งหมด

```

/*****/

/* sub program header define */

/*****/

void move_to_00(void);
void CheckInputAndAction(void);
void move_x_motor(void);
void move_y_motor(void);
void send_outport(void);
void XPosAction(void);
void YPosAction(void);

```

3.3 ส่วน Procedure

ในแต่ละ procedure จะมีคำอธิบายการทำงานของแต่ละ Procedure

```

Procedure name :      mydelay
Description:          Make delay time relate with input value.
Input:                mytimer : long
Return:               none
void mydelay(long mytimer)

```

```

{
    long mydelaytimer;

    mydelaytimer = jiffies;
    while((jiffies - mydelaytimer) < mytimer) ;
}

```

Procedure name : chartoint
Description: Receive character then change to integer value
Input: charin : char
Return: integer value of char input

```

int chartoint(int charin)
{
    int retval;
    switch(charin)
    {
        case '0' : retval = 0; break;
        case '1' : retval = 1; break;
        case '2' : retval = 2; break;
        case '3' : retval = 3; break;
        case '4' : retval = 4; break;
        case '5' : retval = 5; break;
        case '6' : retval = 6; break;
        case '7' : retval = 7; break;
        case '8' : retval = 8; break;
        case '9' : retval = 9; break;
        default: retval = -1;
    }
    return retval;
}

```

Procedure name : `move_x_motor`

Description: `change step index of X stepping motor by increase or decrease index`
 `1 step that related with stepping motor direction variable.`

Input: `xdirect`

Return: `xindex`

```

/* use for move motor in 3 direction */
/* 'r' = reverse direction */
/* 'f' = forward direction */
/* 'n' = none no move */
/* x motor move */
void move_x_motor()
{
    if(xdirect == 'r')
    {
        xindex--;
        /* move x motor backward */
        if(xindex < 0) xindex = MAXSTEP;
    }
    else if(xdirect == 'f')
    {
        xindex++;
        /* move x motor forward */
        if(xindex > MAXSTEP) xindex = 0;
    }
}

```

Procedure name : `move_y_motor`

Description: `change step index of Y stepping motor by increase or decrease index`
 `1 step that related with stepping motor direction variable.`

Input: `ydirect`

Return: `yindex`

```

void move_y_motor()
{
    if(ydirect == 'r')
    {
        yindex--;          /* move y motor backward */
        if(yindex < 0) yindex = MAXSTEP;
    }
    else if(ydirect == 'f')
    {
        yindex++;          /* move y motor forward */
        if(yindex > MAXSTEP) yindex = 0;
    }
}

```

Procedure name : send_outport

Description: send data to both stepping motor only for separating x motor and y motor by using bit mask.

Input: yindex,xindex,ysteptable,xsteptable

Return: none

```

void send_outport()
{
    motorout = 0;
    motorout = ysteptable[yindex] | xsteptable[xindex];
    outb((motorout & 0xFF) , SPPDATAPORT);
}

```

Procedure name : proc_read_position

Description: return crane position to user.

Input: none

Return: xpos,ypos


```

static int proc_read_position(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}

```

Procedure name : `proc_read_status`
Description: `return crane status to user.`
Input: `status pointer`
Return: `status text`

```

static int proc_read_status(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    char *outchar = (char *)data;
    int i=0;
    while((outchar[i] != '\n') && (i<255))
    {
        page[i] = outchar[i];
        i++;
    }
    page[i] = '\n';
    i++;
    return i;
}

```

Procedure name : `proc_move_only`
Description: `Control stepping motor moving.`
Input: `x,y target position`

Return: none

```
static int proc_move_only(struct file *file, const char *buffer,
                          unsigned long count, void *data)
{
    long i;
    int tempx, tempy;
    /*char xdirect,ydirect; */
    /* check data input correct */
    if(count > 3)
    {
        printk("user request position more than 2 digit\n");
        get_status_file->data = user_position_error;      /* set status text to user
mistake */

        goto nothing_to_do;
    }
    else if(count < 3)
    {
        printk("user request position less than 2 digit\n");
        get_status_file->data = user_position_error;      /* set status text to user
mistake */

        goto nothing_to_do;
    }
    else printk("move to x=%c y=%c\n",buffer[0],buffer[1]);

    /*change char to int for compare*/
    tempx = chartoint(buffer[0]);      /*set x target to tempx */
    tempy = chartoint(buffer[1]);      /*set y target to tempy */

    /* check for valid number */
    if((tempx==-1) || (tempy==-1))
    {
```

```

        printk("user not keyin number\n");

        get_status_file->data = user_keyin_error; /* set status text to user mistake */
    }

    else /* every thing ok so we move the motor */
    {

        printk("x and y motor moving\n");
        get_status_file->data = moving_string ;
        if((temp_x==0)&&(temp_y==0)) move_to_00();

        else
        {
            timeout =0;
            xyposchange = 0;
            while(!((temp_x == xpos) && (temp_y == ypos))&&(timeout <
MAXTIMEOUTDELAY)) /* wait for x and y position */
            {

                /* check x position for move */
                if(temp_x < xpos)
                    xdirect = 'r'; /* move reverse direction */
                else if(temp_x > xpos)
                    xdirect = 'f'; /* move forward direction */
                else
                    xdirect = 'n'; /* not move */

                /* check y position for move */
                if(temp_y < ypos)
                    ydirect = 'r'; /* move reverse direction */
                else if(temp_y > ypos)
                    ydirect = 'f'; /* move forward direction */
                else
                    ydirect = 'n'; /* not move */

                move_x_motor();
                move_y_motor();
            }
        }
    }

```

```

        send_outport();
        mydelay(MAXMOTORDELAY);
        timeout ++;
    }
    printk("crane stand by \n");
    get_status_file->data = standby_string ;
}
}
if(timeout >= MAXTIMEOUTDELAY)
{
    get_status_file->data = timeout_error_string ;
    move_to_00();
}
nothing_to_do:
    xyposchange = 1;
    return count;
}

```

Procedure name : XPosAction

Description: Update x position.

Input: xdirect : char keep x direction

Return: xpos : long keep x position

void XPosAction()

```

{
    if((xyposchange==0))                /* not count if motor not moving */
    {
        if(xdirect == 'r')
            xpos -- ;
        if(xdirect == 'f')
            xpos ++;
    }
}

```

Procedure name : YPosAction
 Description: Update y position.
 Input: ydirect : char keep y direction
 Return: ypos : long keep y position

```

void YPosAction()
{
    if(xyposchange==0)
    {
        if(ydirect == 'r')
            ypos -- ;
        if(ydirect == 'f')
            ypos ++;
    }
}
  
```

Procedure name : CheckInputAndAction

Description: Check printer port value.

Input: parallel data

Return: none

```

void CheckInputAndAction()
{
    int ipA;
    outb(SPPPORTREAD|INPUTENABLE,SPPCONTROLPORT);
    ipA = inb(SPPDATAPORT);
    if(ipA & 0x80)                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
}
  
```

```

if(ipA & 0x20)          /* top limit switch */
{
    ydirect = 'n';
    ypos = 6;
}

if(ipA & 0x40)          /*left limit switch */
{
    xdirect = 'n';
    xpos = 0;
}

if(ipA & 0x10)          /* right limit switch */
{
    xdirect = 'n';
    xpos = 6;
}

if(ipA & 0x01)          /* x sensor */
{
    if(xyposchange==0)
    {
        if(xpos==0)
        {
            if(xflag==0)
            {
                xflag = 1;
                xmotorstarttime = jiffies;
                XPosAction();
                printk("x sensor active  jiffies=%d \n",jiffies);
            }
        }
        else

```

```

        {
            if((jiffies-xmotorstarttime) >
MAXSENSORDELAY)

                xflag = 0;
        }
    }
    else
    {
        if(xflag==0)
        {
            xflag = 1;
            xmotorstarttime = jiffies;
        }
        else
        {
            if(((jiffies-xmotorstarttime) >
MINSENSORDELAY) && (xflag2==0))
            {
                xflag2 = 1;
                XPosAction();
                printk("x sensor active  jiffies=%d
\n",jiffies);
            }

            if(((jiffies-xmotorstarttime) >
MAXSENSORDELAY) && (xflag2==1))
            {
                xflag2 = 0;
                xflag = 0;
            }
        }
    }
}

```

```

    }

}

if(ipA & 0x02)                                /* y sensor */
{
    if(xyposchange==0)
    {
        if(ypos==0)
        {
            if(yflag==0)
            {
                yflag = 1;
                ymotorstarttime = jiffies;
                YPosAction();
                printk("y sensor active  jiffies=%d \n",jiffies);
            }
            else
            {
                if((jiffies-ymotorstarttime) >
MAXSENSORDELAY)
                    yflag = 0;
            }
        }
    }
    else
    {
        if(yflag==0)
        {
            yflag = 1;
            ymotorstarttime = jiffies;
        }
    }
    else

```



```

        {
            if(((jiffies-ymotorstarttime) >
MINSENSORDELAY) && (yflag2==0))
            {
                yflag2 = 1;
                YPosAction();
                printk("y sensor active  jiffies=%d
\n",jiffies);
            }

            if(((jiffies-ymotorstarttime) >
MAXSENSORDELAY) && (yflag2==1))
            {
                yflag2 = 0;
                yflag = 0;
            }
        }
    }
}

outb(SPPINTERRUPTENABLE|OUTPUTENABLE|OUTPUTLATCH,SPPCONTROLPOR
T);
}

```

Procedure name : move_to_00

Description: control stepping motor for bring crane to 0,0 position.

Input: none

Return: none

```
void move_to_00(void)
```

```
{
```

```

        long i;
/* initial crane position to -1 */
        xdirect = 'r';
        ydirect = 'r';
        timeout = 0;
        CheckInputAndAction();
        while(((xdirect != 'n') || (ydirect != 'n')) && (timeout < MAXTIMEOUTDELAY) )
        {
                move_x_motor();
                move_y_motor();
                send_outport();
                mydelay(MAXMOTORDELAY);
                timeout ++;
        }
}

```

Procedure name : crane_interrupt_interrupt

Description: This procedure action when interrupt occur.

Input: none

Return: none

```

void crane_interrupt_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
        CheckInputAndAction();
}

```

Procedure name : init_crane_controller

Description: Prepare all process file and initialize all require variables.

Input: none

Return: error code : upon error case

```

static int __init init_crane_controller(void)

```

```

{

int rv = 0;

int i;

/* Create the crane_controller /proc entry */
main_dir = proc_mkdir("crane_controller", NULL);
if(main_dir == NULL) {
    rv = -ENOMEM;
    goto out;
}

main_dir->owner = THIS_MODULE;

/* Create liftacmains and make it readable by all - 0444 */
move_only_file = create_proc_entry("move_only", 0666, main_dir);
if(move_only_file == NULL) {
    rv = -ENOMEM;
    goto no_move_only;
}

move_only_file->read_proc = NULL;
move_only_file->write_proc = &proc_move_only;
move_only_file->owner = THIS_MODULE;

    /*status file */
get_status_file = create_proc_entry("crane_status", 0444, main_dir);
if(get_status_file == NULL) {
    rv = -ENOMEM;
    goto no_get_status;
}

get_status_file->data = init_string;

```

```

get_status_file->read_proc = &proc_read_status;
get_status_file->write_proc = NULL;
get_status_file->owner = THIS_MODULE;

    /*position file */
get_position_file = create_proc_entry("crane_position", 0444, main_dir);
if(get_position_file == NULL) {
    rv = -ENOMEM;
    goto no_get_position;
}

get_position_file->read_proc = &proc_read_position;
get_position_file->write_proc = NULL;
get_position_file->owner = THIS_MODULE;

crane_interrupt_file = create_proc_entry("crane_interrupt", 0444, main_dir);
if(crane_interrupt_file == NULL) {
    return -ENOMEM;
}

crane_interrupt_file->read_proc = NULL;
crane_interrupt_file->write_proc = NULL;
crane_interrupt_file->owner = THIS_MODULE;

/* request interrupt from linux */
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,
    "crane_interrupt",NULL);
if ( rv ) {
    printk("Can't get interrupt %d\n", INTERRUPT);
    goto no_interrupt;
}

```

```

        move_to_00());
    if(timeout >= MAXTIMEOUTDELAY)
    {
        printk("timeout = %d , xdirect = %c , ydirect = %c
\n",timeout,xdirect,ydirect);
        printk("Could not detect crane controller board. Program running in demo
mode\n");
        get_status_file->data = demo_string;
    }
    else
    {
        printk("timeout = %d , xdirect = %c , ydirect = %c
\n",timeout,xdirect,ydirect);
        printk("Program running in real time mode\n");
        get_status_file->data = standby_string ;
    }

/* everything initialed */
printk(KERN_INFO "%s %s initialized\n", MODULE_NAME, MODULE_VERSION);
return 0;

/* this removes /proc entries if we have an error along the way */
no_interrupt:
    remove_proc_entry("crane_interrupt",main_dir);
no_get_position:
    remove_proc_entry("crane_position",main_dir);
no_get_status:
    remove_proc_entry("crane_status",main_dir);
no_move_only:
    remove_proc_entry("move_only", main_dir);

out:    return rv; }

```

Procedure name : cleanup_crane_controller

Description: Clear all values that init maked.

Input: none

Return: none

```
static void __exit cleanup_crane_controller(void)
{
    free_irq(INTERRUPT,NULL);

    /* removing the /proc entries */
    remove_proc_entry("crane_interrupt",main_dir);
    remove_proc_entry("crane_position",main_dir);
    remove_proc_entry("crane_status",main_dir);
    remove_proc_entry("move_only", main_dir);
    remove_proc_entry("crane_controller", NULL);

    /* we're done */
    printk(KERN_INFO "%s %s removed\n", MODULE_NAME, MODULE_VERSION);
}
```

ส่วนสุดท้ายเป็นวิธีการเรียกใช้งาน initial procedure และ cleanup procedure

```
module_init(init_crane_controller);
module_exit(cleanup_crane_controller);

EXPORT_NO_SYMBOLS;
```

4. การ compile code

การ compile source code จะต้องทำแบบ module compile โดยการพิมพ์คำสั่งดังนี้

```
gcc -DMODULE -D__KERNEL__ -c crane_controller.c
```

จะได้ file crane_controller.o ซึ่งเป็น Linux module เพื่อใช้ในการ insert เข้า kernel ต่อไป

5. การ run /stop program

การสั่งให้ program ทำงานทำได้โดยพิมพ์คำสั่งดังนี้

```
insmod crane_controller.o
```

การตรวจสอบว่า module ทำงานอยู่หรือไม่พิมพ์คำสั่งดังนี้

```
lsmod
```

การสั่งให้ program หยุดทำงาน

```
rmmod crane_controller
```

6. การทำ Startup script

ใน Debian Linux สามารถทำ startup script ได้โดยนำ shell script ไปใส่ไว้ยังตำแหน่งต่อไปนี้

```
/etc/rc.S
```

Directory นี้จะเก็บ startup script ของระบบทั้งหมด โดยลำดับการทำงานของแต่ละ script จะจัดลำดับโดยการตั้งชื่อ script เช่น S01INIT จะถูก run ก่อน S30MYSHELL เป็นต้น